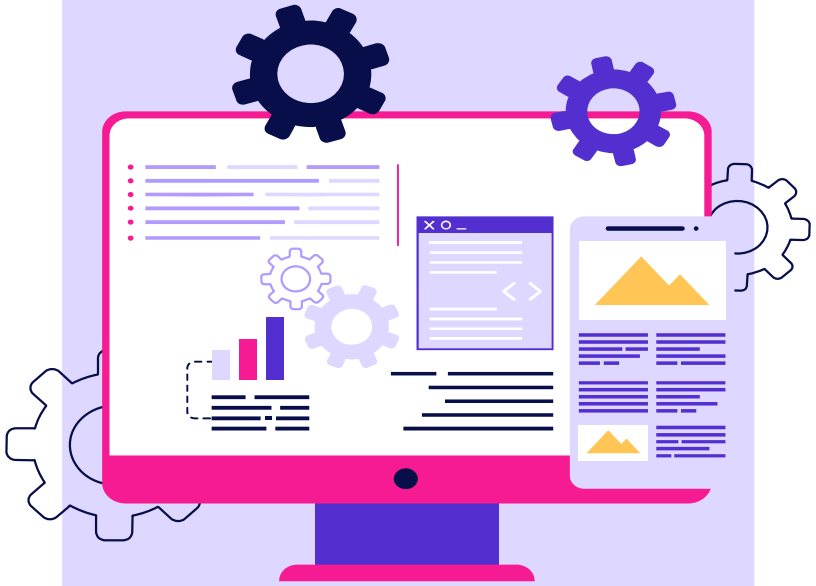
ED2 – Métodos de Ordenação

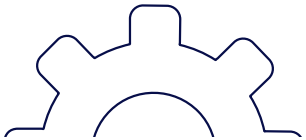
Profª Juliana Franciscani



Métodos de Ordenação Eficiente

Usados para grandes volumes de dados:

- Shell Sort
- Merge Sort
- Quick Sort
- Heap Sort



Shell Sort

- **Função shellSort:**

Função que inicia com um gap grande (metade do tamanho do vetor) e vai diminuindo o valor do gap progressivamente até que ele seja 1. Para cada valor de gap, o algoritmo realiza uma ordenação por inserção, mas de forma que os elementos com um intervalo de gap entre eles sejam comparados e trocados.

Vídeo Dança

Explicação

Passo 1) Tamanho do vetor é 8. Gap será h = 8/2 ou 4.

Passo 2) Armazena dados a uma distância de 4. Sublistas: {8, 1}, {6, 4}, {7, 5}, {2, 3}.

Passo 3) Sublistas serão classificadas usando classificação por inserção, onde uma variável temporária é usada para comparar os dois números e classificar.

Passo 4) Diminui valor inicial do intervalo. O novo intervalo será h=h/2 -> 4/2 -> 2.

Passo 5) h é > 0, volta ao **passo 2** com gap de 2. Sublistas {1, 5, 8, 7}, {4, 2, 6, 3}

Passo 6) Sublistas serão classificadas usando classificação por inserção.

Passo 7) Diminui intervalo, h=h/2 -> 2/1 -> 1. Funciona como inserção.

Exemplo de ordenação utilizando Shell Sort

3	5	2	8	7	4	1	6
0	1	2	3	4	5	6	7

Etapa 1: GAP 8/2=4

3	5	2	8	7	4	1	6

3	5	2	8	7	4	1	6

3	4	2	8	7	5	1	6

3	4	1	8	7	5	2	6

3	4	1	6	7	5	2	8

Etapa 2: GAP 4/2 = 2

3	4	1	6	7	5	2	8

1	4	3	6	7	5	2	8

1	4	3	6	7	5	2	8

1	4	3	5	7	6	2	8

1	4	2	5	3	6	7	8

1	4	2	5	3	6	7	8

1	4	2	5	3	6	7	8

Etapa 3: GAP 2/2 = 1 → Insertion

Exemplo de ordenação utilizando Shell Sort

Etapa 3: GAP 2/2 = 1 → Insertion

1	4	2	5	3	6	7	8
0	1	2	3	4	5	6	7

1	4	2	5	3	6	7	8

1	4	2	5	3	6	7	8

1	4	2	5	3	6	7	8

1	2	4	5	3	6	7	8

1	2	4	5	3	6	7	8

1	2	4	5	3	6	7	8

1	2	4	5	3	6	7	8

1	2	4	5	5	6	7	8

1	2	4	4	5	6	7	8

1	2	3	4	5	6	7	8

1	2	3	4	5	6	7	8

1	2	3	4	5	6	7	8

1	2	3	4	5	6	7	8

1	2	3	4	5	6	7	8

1	2	3	4	5	6	7	8

```
1.  void metodoShell (int vetor[], int tam){
2.      int i, j, aux, gap;
3.      for (gap=tam/2; gap>0; gap/=2){
4.          for (i=gap; i<tam; i++){
5.              aux = vetor[i];
6.              j=i;
7.              while (j>=gap && vetor[j-gap]>aux){
8.                vetor[j] = vetor[j-gap];
9.                j-=gap;
10.             }
11.             vetor[j] = aux;
12.         }
13.     }
14. }
```

```
void metodoShell(int vetor[], int tam) {
    int i, j, aux, gap;

    // Inicializa o gap e diminui até 1 (vira insertion)
    for (gap = tam / 2; gap > 0; gap /= 2) {
        for (i = gap; i < tam; i++) {
            aux = vetor[i];
            j = i;
           //while para voltar no vetor e comparar
           // comparação com os anteriores é necessário
            while (j >= gap && vetor[j - gap] > aux) {
                vetor[j] = vetor[j - gap];
                j -= gap;
            }
            vetor[j] = aux;
        }
    }
}
```

Merge Sort

- Divisão e Conquista:

O Merge Sort utiliza a técnica de divisão e conquista para dividir o vetor recursivamente até que os subvetores tenham apenas um elemento, e então os mescla de volta de forma ordenada.

- Utiliza duas funções: Merge() e MergeSort()

Vídeo Dança

Merge Sort

- **Função mergeSort:**

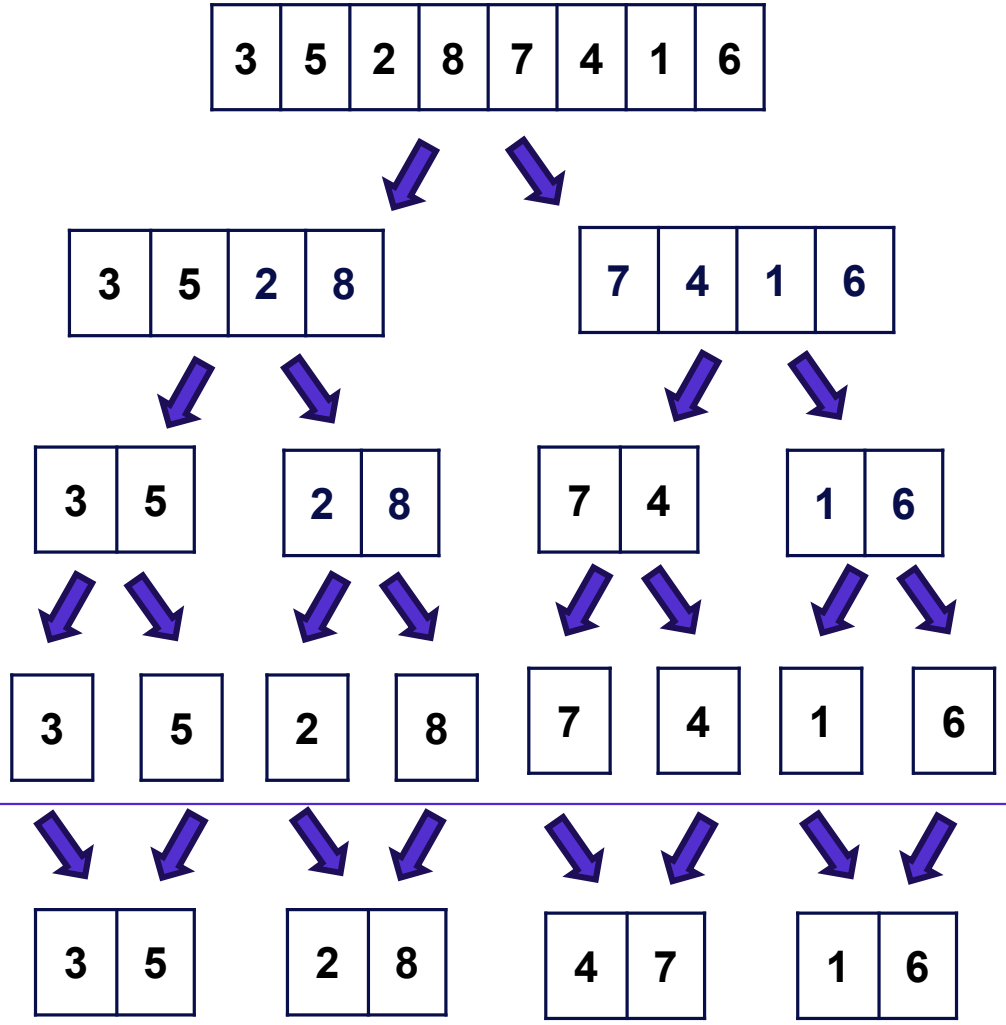
Função recursiva divide o vetor ao meio, ordena as duas metades e, em seguida, as mescla de volta com a função merge.

- **Função merge:**

A função de mesclagem é responsável por unir as duas metades ordenadas em um único vetor ordenado. Intercala os dados de forma ordenada em vetor maior

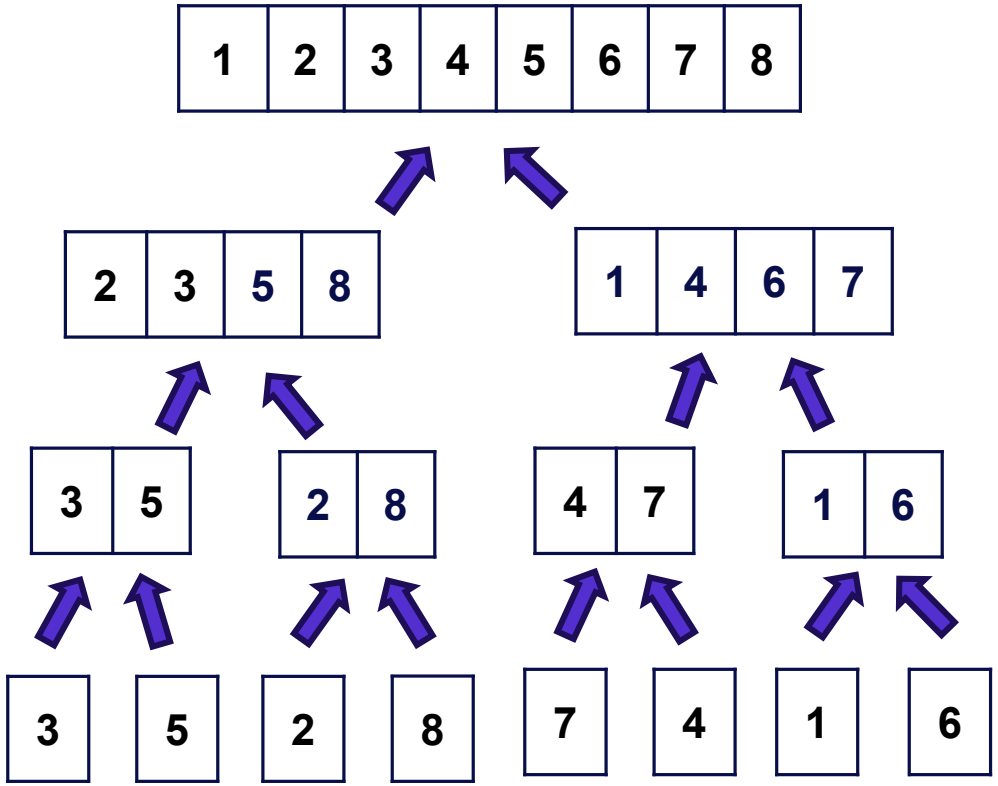
Exemplo de ordenação utilizando Merge Sort

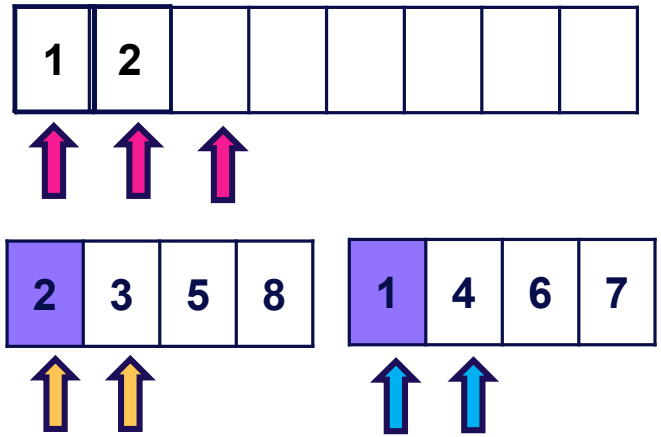
1º chama a função MERGESORT Dividir até chegar em blocos unitários



MERGE Ordenada

MERGE - Ordena os subvetores até chegar no tamanho real





2<1 não,
insere 1 no v
incrementa índice de v2

2<4 sim,
Insere 2 no v
incrementa índice de v1

Algoritmo Merge Sort

```
int mergeSort(int *vetor, int inicio, int fim) {
    if(inicio >= fim) return ;

    int meio = inicio+(fim-inicio)/2;//calculo do meio
    mergeSort(V,inicio,meio);
    mergeSort(V,meio+1,fim);
    merge(V,inicio,meio,fim);
}
```

Chama a função para cada metade

Combina as metades de forma ordenada

```
void merge(int *V, int inicio, int meio, int fim){
    int *temp, tam;
    tam = fim-inicio+1;
    temp = new int [tam];
    int p1, p2, i, j, fim1 = 0, fim2 = 0;
    p1 = inicio; p2 = meio+1;
```

Combinar ordenando

Vetor acabou?

Copia o que sobrar

Copiar do auxiliar para o original

```
    if(temp != NULL){
        for(int i=0; i<tam; i++){
            if(fim1==0 && fim2==0){//if(!fim1 && !fim2)
                if(V[p1] < V[p2]){
                    temp[i]=V[p1];//temp[i]=V[p1++];
                    p1++;
                }else{
                    temp[i]=V[p2];
                    p2++;
                }
                if(p1>meio) fim1=1;
                if(p2>fim) fim2=1;
            }else{
                if(fim1==0)
                    temp[i]=V[p1++];
                else
                    temp[i]=V[p2++];
            }
        }
        for(j=0, k=inicio; j<tamanho; j++, k++)
            V[k]=temp[j];
    }
    delete[]temp;
}
```

Exercícios

10	3	5	13	2	6	11	0	8	4	14	1	7	9

10	3	2	1	5	6	11	7	8	4	9

Quick Sort

- **Função partition:**

O pivô é escolhido como o último elemento do vetor. O vetor é percorrido e os elementos menores que o pivô são movidos para a esquerda, e os maiores para a direita. Quando a partição é feita, o pivô é colocado na posição correta no vetor.

- **Função quickSort:**

A função recursiva divide o vetor em duas subpartes: uma à esquerda do pivô e outra à direita. A partição é feita e as duas subpartes são ordenadas recursivamente.

Vídeo Dança

Explicação

- Um elemento é escolhido como pivô
- Valores menores do que o pivô são colocados antes dele e os maiores, depois
- Supondo o pivô na posição **X**, esse processo cria duas partições: **[0,...,X-1]** e **[X+1,...,N-1]**.
- Aplicar o processo recursivamente a cada partição. Até que cada partição contenha um único elemento.

Explicação

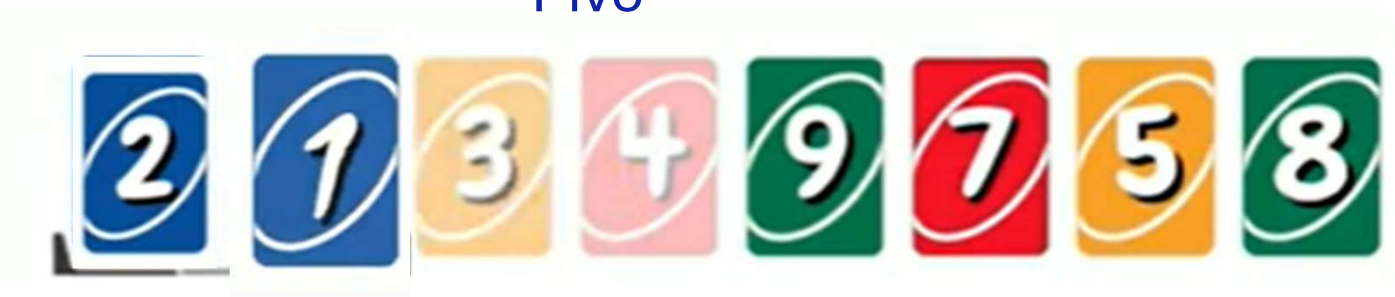
- Algoritmo usa 2 funções:
  - quickSort : divide os dados em arrays cada vez menores
  - particiona: calcula o pivô e rearranja os dados

Passo a passo

Pivô como último elemento

Passo a passo

Pivô



Pivô

Passo a passo

Pivô

Passo a passo

Pivô

Pivô

Pivô

Função quickSort()

```
void quickSort(int *V, int inicio, int fim) {
    int pivo;
    if(fim > inicio){
        pivo = particiona(V, inicio, fim);
        quickSort(V, inicio, pivo-1);
        quickSort(V, pivo+1, fim);
    }
}
```

Separa os dados em 2 partições

Chama a função para as 2 metades

Função particiona()

```
int particiona(int *V, int inicio, int final ){
    int esq, dir, pivo, aux;
    esq = inicio;
    dir = final;
    pivo = V[inicio];
    while(esq < dir){
        while(esq <= final && V[esq] <= pivo)
            esq++;

        while(dir >= 0 && V[dir] > pivo)
            dir--;

        if(esq < dir){
            aux = V[esq];
            V[esq] = V[dir];
            V[dir] = aux;
        }
    }
    V[inicio] = V[dir];
    V[dir] = pivo;
    return dir;
}
```

Avança posição da esquerda

Recua posição da direita

Trocar esq e dir

Passo a passo

Pivô como primeiro elemento

Particiona(V,0,6)

Pivo=v[0]; → 4
esq<=pivô? Sim.
Incrementa esq
esq<=pivô? Não.
Comparar com dir
dir < pivô? Sim.
Trocar esq e dir de lugar

esq< dir? Sim.
Continua o while

esq<=pivô? Sim.
Incrementa esq

esq<=pivô? Sim.
Incrementa esq

Passo a passo

Pivô como primeiro elemento

Pivo=v[0]; → 4
esq<=pivô? Não
Comparar com dir

dir < pivô? Não.
Decrementa dir

dir < pivô? Não.
Decrementa dir

dir < pivô? Não.
Decrementa dir

dir < pivô? Sim.
Esq<dir? não
Termina o while

Troca o pivô com dir

Passo a passo

Particiona(V,0,2)

Particiona(V,4,6)

Resolução a partir de um exemplo

5	3	7	2	1	8	4

MS (V, 0, 6){V= [5,3,7,2,1,8,4]
i=0, f=6
M=0+3=3;
E: MS(V, 0, 3){ V= [5,3,7,2]
i=0, f=3
M=0+1=1;
E: MS(V, 0, 1){ V[5,3]
i=0, f=1;
Meio = 0
E: MS(V, 0, 0){ V=[5]
i=0, f=0;
Sai;
}
D: MS(V,1,1){ V=[3]
i=1, f=1;
Sai;
}
Merge() => V=[3,5]
}

D: MS(V, 2, 3){ => V[7,2]
i=2, f=3;
Meio = 2
E: MS(V, 2, 3){ V=[7]
i=3, f=3;
Sai;
}
D: MS(V,3,3){ V=[2]
i=1, f=1;
Sai;
}
Merge() => V=[2,7]
}
Merge() => V=[2,3,5,7]
}

D: MS(V, 4, 6){ V= [1,8,3]
i=4, f=6
M=4+1=5;

E: MS(V, 4, 5){ : V[1,8]
i=4, f=5;
Meio = 4+0
E: MS(V, 4,4) { : V[1]
i=4, f=4
Sai x
}
D: MS(V, 5,5) {: V[8]
i=5, f=5
Sai x
}
Merge() => V=[1,8]
}

D: MS(V, 6, 6){ => V[4]
i=6, f=6;
Sai x
}
Merge() => V=[1, 4, 8]
}
Merge () => V=[1, 2, 3, 4, 5, 7, 8]
}

```
MS(V,I,F){
    Se I<F sai.
    m= I+ (F-I)/2
    MS (V, I, m);
    MS(V, m+1, F);
    Merge();
}
```

Heap Sort

- **Função heapify:**

Usada para garantir que a propriedade do heap seja mantida. No caso do max heap, ela garante que o valor na raiz seja o maior entre o nó e seus filhos. Se a raiz não for o maior, a função troca a raiz com o maior filho e aplica heapify recursivamente na subárvore.

- **Função heapSort:**

Constrói o heap a partir do vetor, começando do meio do vetor e chamando heapify para cada elemento.

Remove o maior elemento (a raiz) do heap, colocando-o na posição correta do vetor. Reordena o heap para garantir que a propriedade do heap seja mantida.

Referências

EDELWEISS, N,; GALANTE, R.. Estruturas de dados. Porto Alegre: Bookman, 2009.

SZWARCFITER, J. L.; MARKENZON, L. Estruturas de dados e seus algoritmos. 3. ed. Rio de Janeiro: LTC, 2010.

ZIVIANI, N. Projeto de algoritmos: com implementações em Pascal e C. 3. ed. rev. e ampl. São Paulo: Cengage Learning, 2011.

Aulas e vídeo aulas do professor André Backes: https://www.facom.ufu.br/~backes/